Android: Basics on development

Desenvolvimento de Software e Sistemas Móveis (DSSMV)
Licenciatura em Engenharia de Telecomunicações e Informática
LETI/ISEP

2026/27

Paulo Baltarejo Sousa and Carlos Filipe Freitas
{pbs,caf}@isep.ipp.pt



Disclaimer

Material and Slides

Some of the material/slides are adapted from various:

- Presentations found on the internet;
- Books;
- Web sites;
- ...

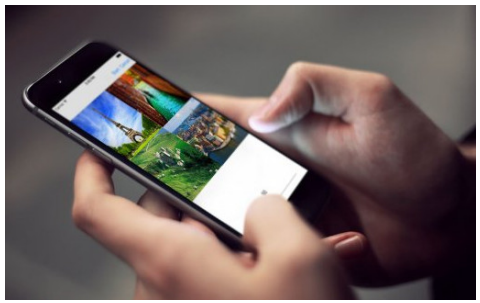
Outline

1. Activities
2. UI Events
3. Intents
4. Shared Preferences
5. Android Adapter
6. Threads & WorkerThreads

Activities

Activity

- An **activity provides a user interface** (UI) for a single screen in an app



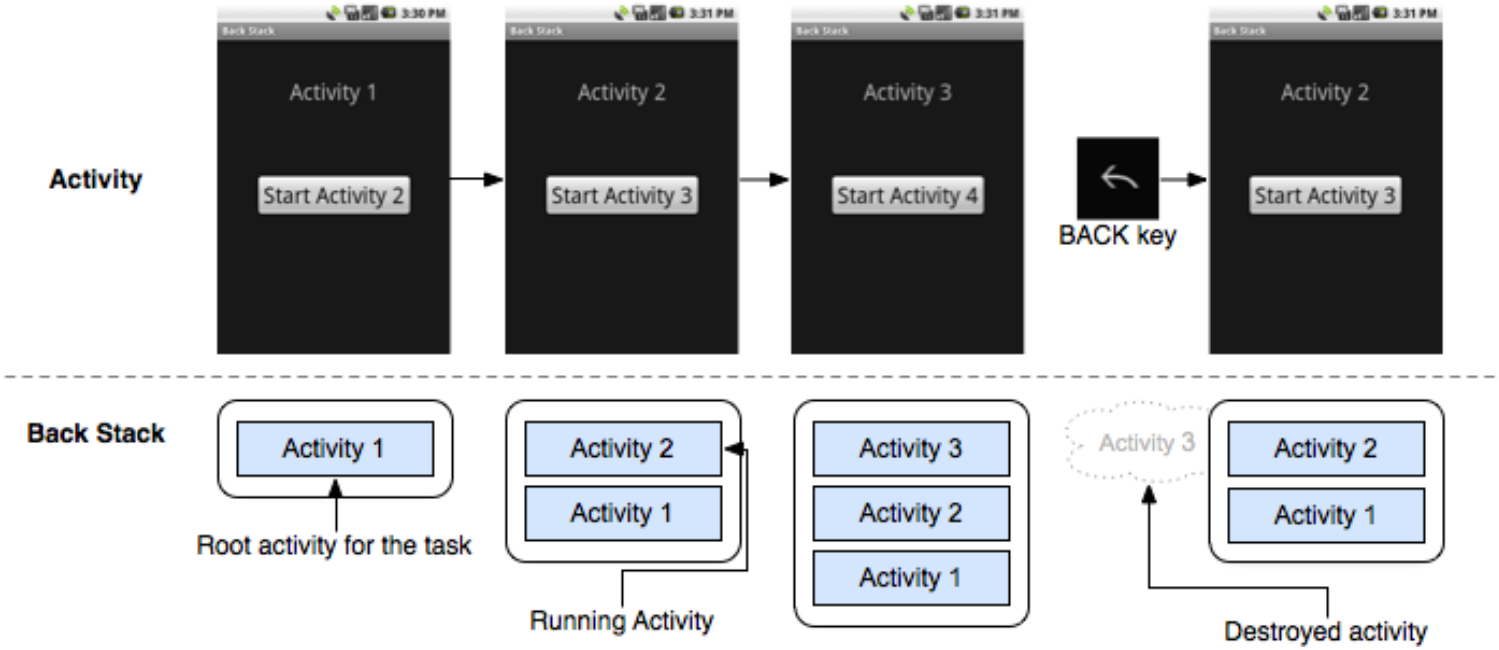
- Every app **activities have to be registered** in the `AndroidManifest.xml` file

```
?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
        package="com.example.listview">

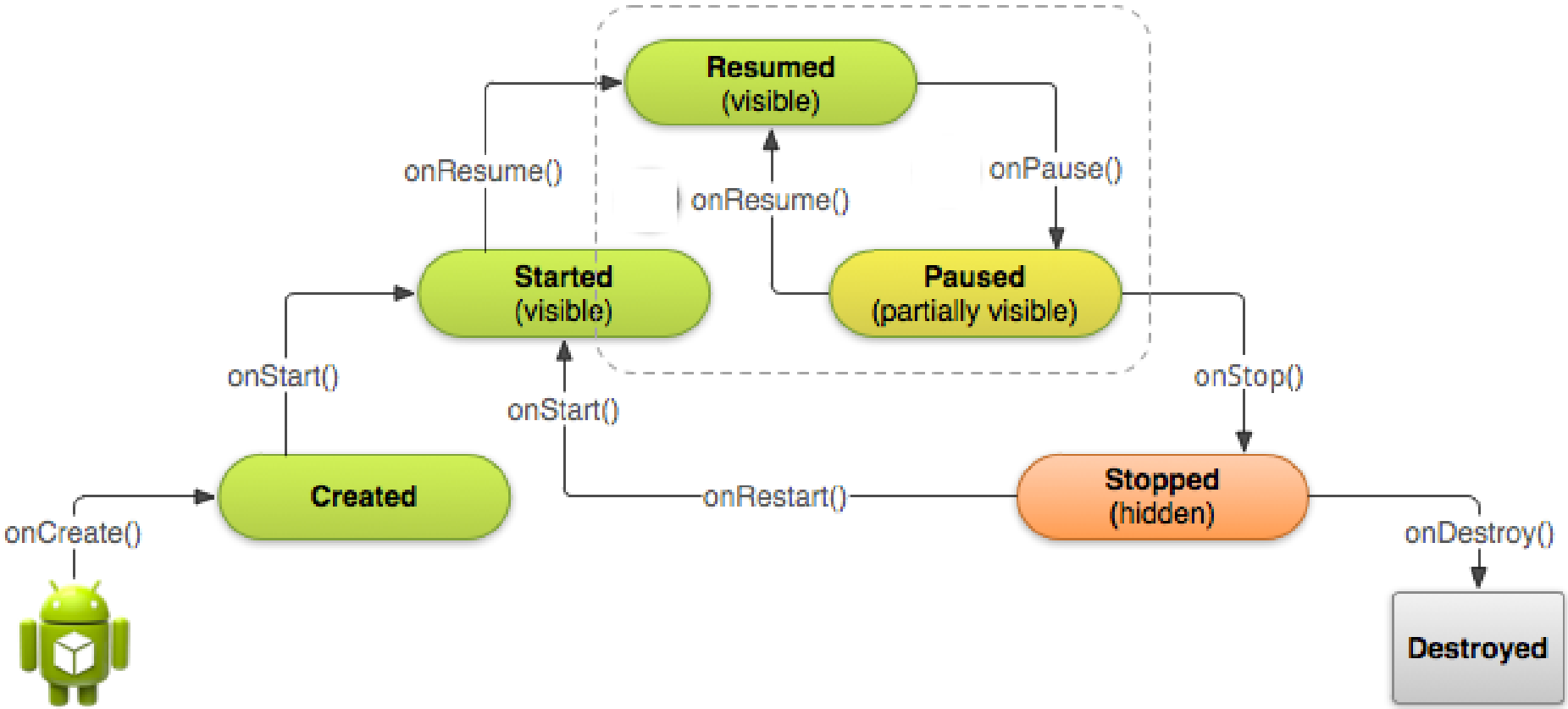
   <application
        ....
      <activity android:name=".MainActivity">
        ...
      </activity>
   </application>
</manifest>
```

Back Stack

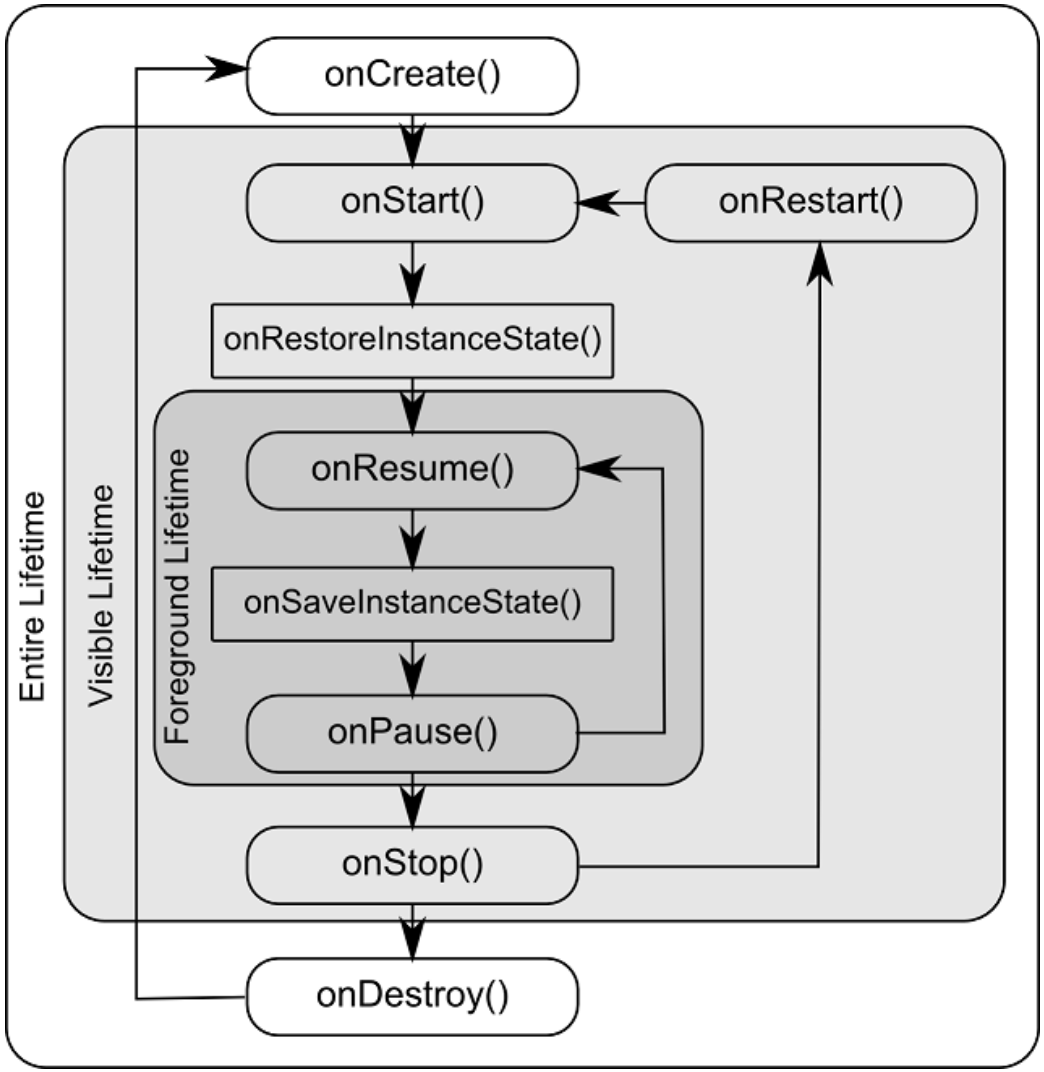
- **Activities are arranged in a stack** (Back Stack), in the order in which each activity is opened.



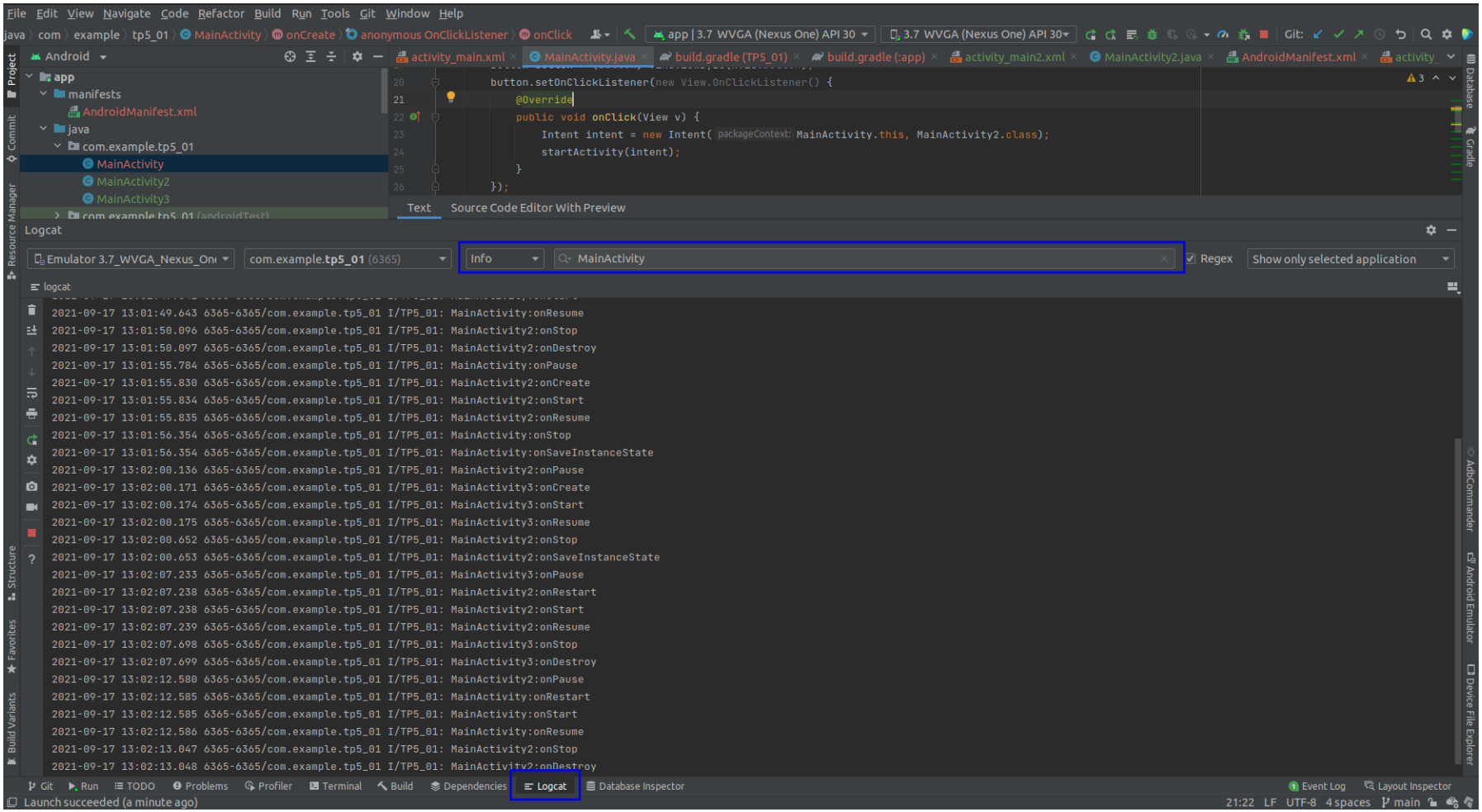
Activity States



Activity Lifecycle (I)



Activity Lifecycle (II)

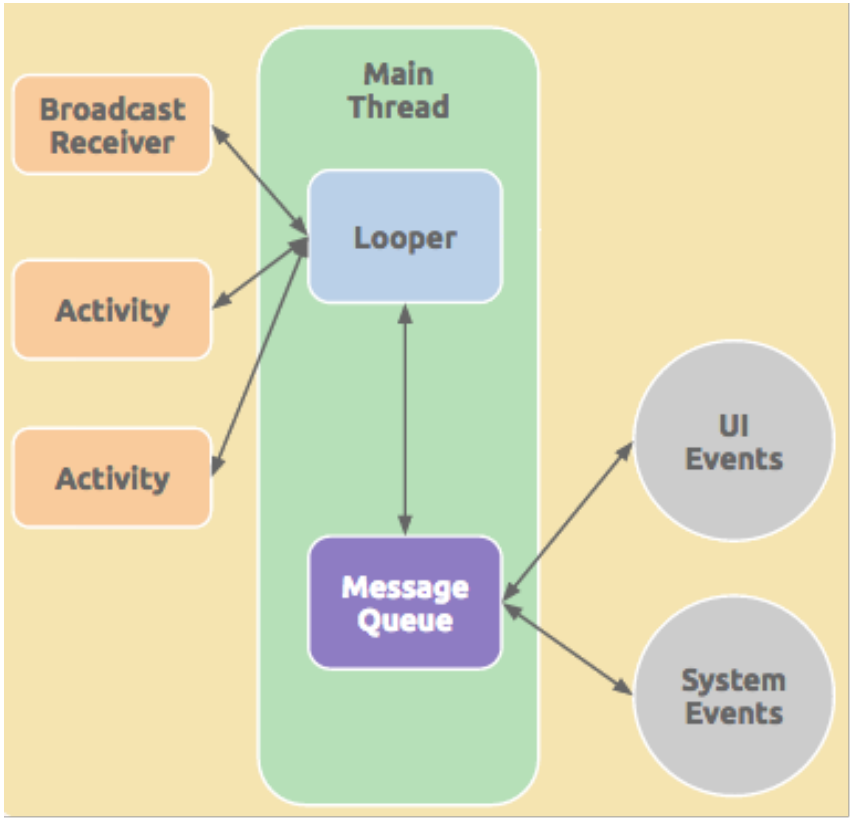


Check: `TP04_01`

UI Events

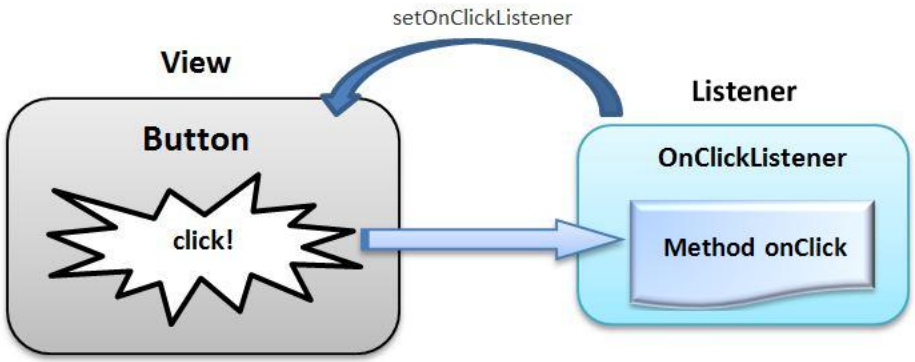
Handling events and messages

- The Android framework maintains an event/message queue into which events are placed as they occur.
- Events/messages are removed from the queue on a first-in, first-out (FIFO) basis.



Event listeners

- **Event Listeners**
  - It is an interface in the `View` class that contains a callback method.
  - Callback method is invoked when the `View` is triggered by user interaction.
- **Event Listeners Registration**
  - It is the process by which an Event Handler gets registered with an Event Listener so that the handler is called when the Event Listener fires the event.
- **Event Handlers**
  - It is the callback method that actually handles the event.



Event Listeners (I)

- `onClick()`
  - From `View.OnClickListener`. This is called when the user either touches the item (when in touch mode), or focuses upon the item with the navigation-keys or trackball and presses the suitable "enter" key or presses down on the trackball.
- `onLongClick()`
  - From `View.OnLongClickListener`. This is called when the user either touches and holds the item (when in touch mode), or focuses upon the item with the navigation-keys or trackball and presses and holds the suitable "enter" key or presses and holds down on the trackball (for one second).
- `onFocusChange()`
  - `View.OnFocusChangeListener`. This is called when the user navigates onto or away from the item, using the navigation-keys or trackball.

Event Listeners (II)

- `onKey()`
  - From `View.OnKeyListener`. This is called when the user is focused on the item and presses or releases a hardware key on the device.
- `onTouch()`
  - From `View.OnTouchListener`. This is called when the user performs an action qualified as a touch event, including a press, a release, or any movement gesture on the screen (within the bounds of the item).

```
@Override
protected void onCreate(Bundle savedInstanceState) {
   super.onCreate(savedInstanceState);
   setContentView(R.layout.activity_main);
   Button bt = (Button) findViewById(R.id.button);
   bt.setOnClickListener(new View.OnClickListener() {
      @Override
      public void onClick(View v) {
         ...
      }
   });
}
```

Event Listeners: `AdapterView`

- `OnItemClickListener()`
  - From `AdapterView.OnItemClickListener`. This is called when when an item in this AdapterView has been clicked.
- `OnItemLongClickListener()`
  - From `AdapterView.OnItemLongClickListener`. This is called when an item in this view has been clicked and held.
- `OnItemSelectedListener`
  - From `AdapterView.OnItemSelectedListener`. This is called when an item in this view has been selected.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
   super.onCreate(savedInstanceState);
   setContentView(R.layout.activity_main);
   ListView lv = (ListView) findViewById (R.id.listView);
   lv.setOnItemClickListener(new AdapterView.OnItemClickListener() {
     @Override
     public void onItemClick(AdapterView<?> adapterView, View view, int i, long l) {
       ...
    }
   });
}
```

Event Listeners: `TextWatcher`

- Basically, a `TextWatcher` is used to keep watch on `EditText` while entering data into it.
- We can perform operation and keep watch on which characters are being entered or how many characters are being entered in the `EditText`.

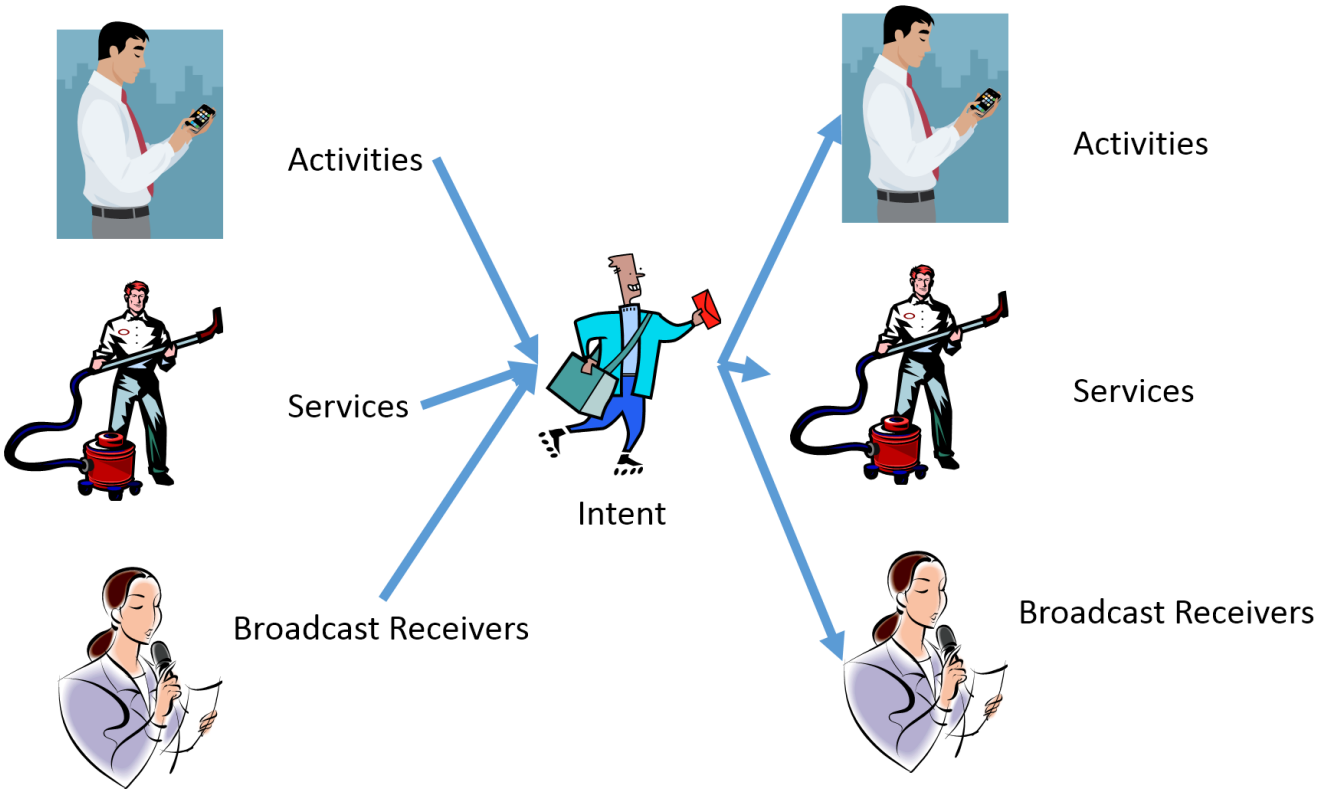
```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_search);

    EditText et = (EditText)findViewById(R.id.editText1);
    et.setText("");
    et.addTextChangedListener(new TextWatcher() {
        @Override
        void beforeTextChanged(CharSequence s, int start, int count, int after) {...}
        @Override
        public void onTextChanged(CharSequence s, int start, int before, int count)
              {...}
        @Override
        public void afterTextChanged(Editable s) {...}
    });
}
```

Intents

What is an Intent

- An Intent is a messaging object you can use to request an action from another app component.
- Intents facilitate communication between components in several ways.



Intents are used for

- To start an activity:
  - The Intent describes the activity to start and carries any necessary data and is passed to `startActivity` method.
  - If you want to receive a result from the activity when it finishes, call `startActivityForResult`. Your activity receives the result as a separate Intent object in your activity's `onActivityResult` callback.
- To start a service:
  - A Service is a component that performs operations in the background without a user interface. You can start a service by passing an Intent to `startService`. The Intent describes the service to start and carries any necessary data.
- To deliver a broadcast:
  - A broadcast is a message that any app can receive. The system delivers various broadcasts for system events, such as when the system boots up or the device starts charging. You can deliver a broadcast to other apps by passing an Intent to `sendBroadcast`.

What is in an Intent

- An Intent object carries information that the Android system uses to determine which component to start, plus information that the recipient component uses in order to properly perform the action.
- The primary information contained in an Intent is the following:
  - **Component name**: The name of the component to start.
  - **Action**: A string that specifies the generic action to perform (such as view or pick).
  - **Data**: The URI (a Uri object) that references the data to be acted on and/or the MIME type of that data.
  - **Category**: A string containing additional information about the kind of component that should handle the intent.
  - **Extras**: Key-value pairs that carry additional information required to accomplish the requested action.
  - **Flags**: Flags defined in the Intent class that function as metadata for the intent.

Intent Types

- **Implicit intents**
  - Do not name a specific component, but instead declare a general action to perform, which allows a component from another app to handle it.
  - For example, if you want to show the user a location on a map, you can use an implicit intent to request that another capable app show a specified location on a map.
- **Explicit intents**
  - Specify the component to start by name (the fully-qualified class name).
  - Used to start a component in your own app, because you know the class name of the activity or service you want to start.

Implicit Intent

- An **implicit intent** specifies an action that can invoke any app on the device able to perform the action.
- Using an implicit intent is useful when your app cannot perform the action, but other apps probably can and you'd like the user to pick which app to use.
  - For example, if you have content you want the user to share with other people, create an intent with the `ACTION_SEND` action and add extras that specify the content to share. When you call `startActivity` with that intent, the user can pick an app through which to share the content.

```
// Create the text message with a string
Intent sendIntent = new Intent();
sendIntent.setAction(Intent.ACTION_SEND);
sendIntent.putExtra(Intent.EXTRA_TEXT, textMessage);
sendIntent.setType("text/plain");

// Verify that the intent will resolve to an activity
if (sendIntent.resolveActivity(getPackageManager()) != null) {
   startActivity(sendIntent);
}
```

Explicit Intent

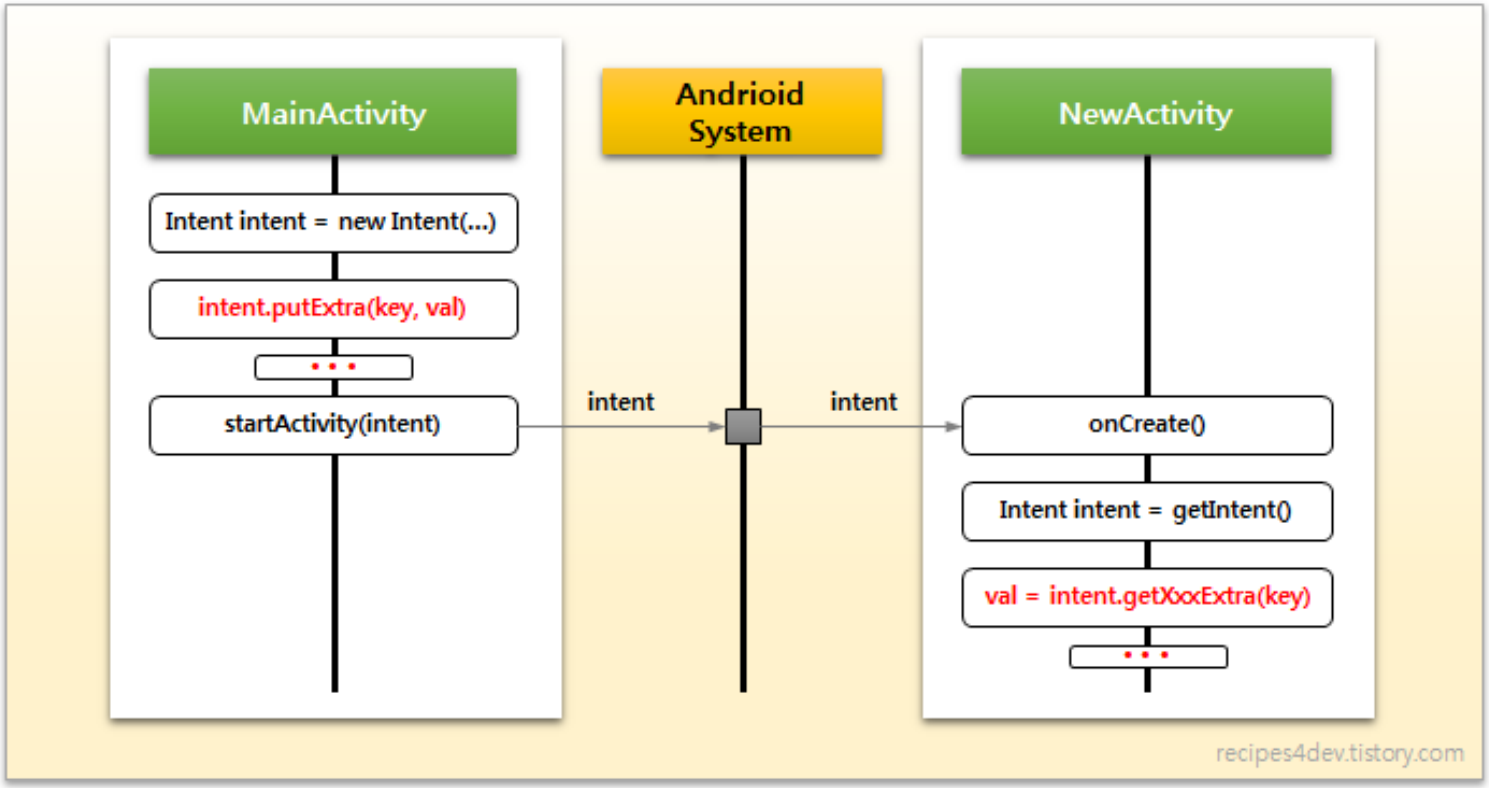
- An **explicit intent** is one that you use to launch a specific app component, such as a particular activity or service in your app. To create an explicit intent, define the component name for the Intent object – all other intent properties are optional.

```
 intent = new Intent(MainActivity.this, SearchActivity.class);
intent.putStringArrayListExtra("LIST",list);
startActivity(intent);
```

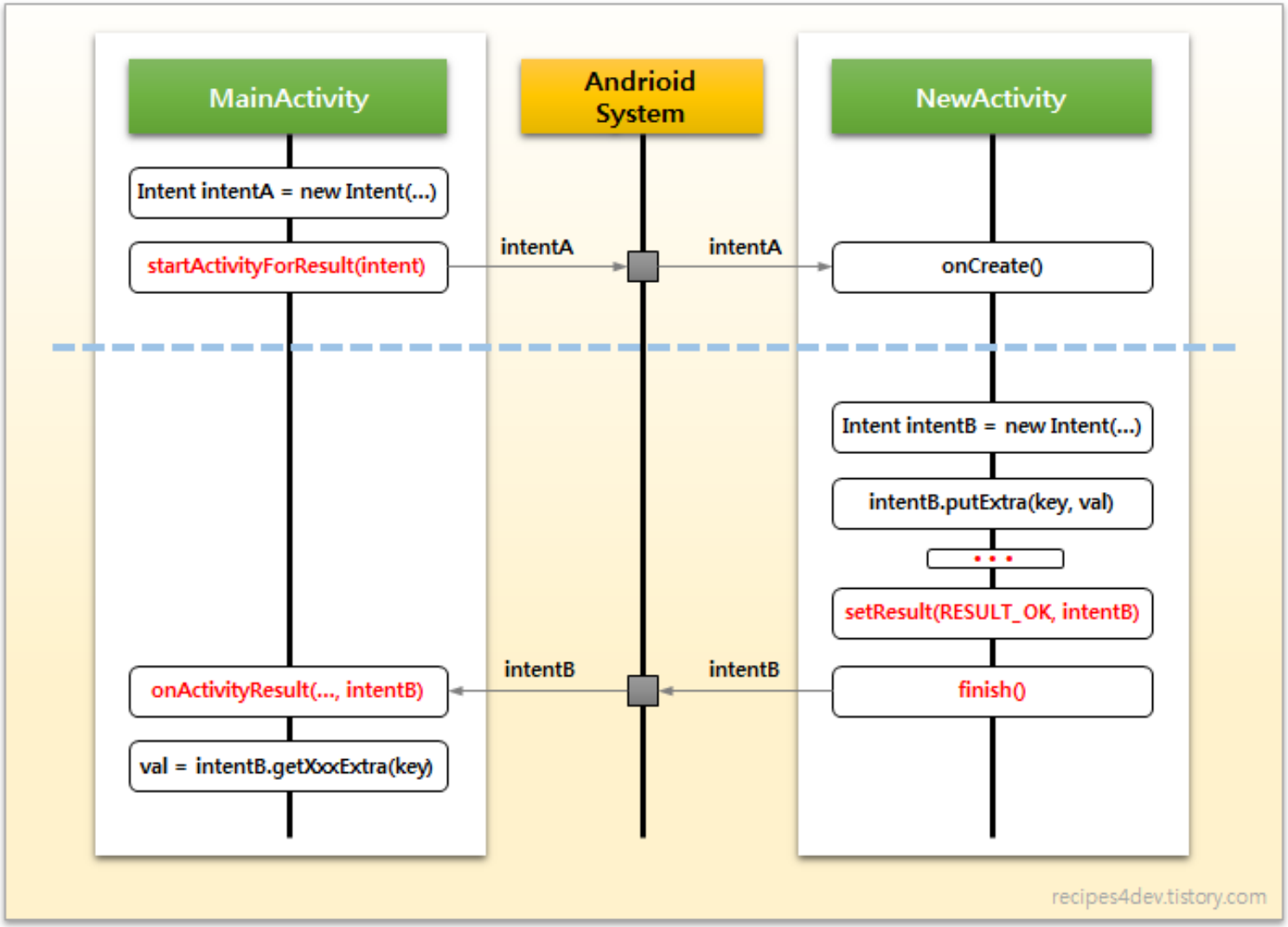
- The `Intent(Context, Class)` constructor supplies the app `Context` and the component a `Class` object.

Intent: data transfer

- `putExtra (Key, Value)`
  - Add extended data to the intent.
- `GetXxxxExtra (Key)`
  - Retrieve data from the intent.



Intent: data returning (I)



Intent: data returning (II)

```
public class MainActivity extends AppCompatActivity {
   static final int REQUEST_CODE_MAIN3ACTIVITY = 1;
   static final int REQUEST_CODE_MAIN4ACTIVITY = 2;
   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
      ...
      Button bt3 = (Button) findViewById(R.id.button3);
      bt3.setOnClickListener(new View.OnClickListener() {
         @Override
         public void onClick(View v) {
            Intent intent = new Intent(MainActivity.this, Main4Activity.class);
            startActivityForResult(intent,REQUEST_CODE_MAIN4ACTIVITY);
         }
      });
    }

}
```

Intent: data returning (III)

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data)
    {
        super.onActivityResult(requestCode, resultCode, data);
        TextView tv = (TextView) findViewById(R.id.textView);
        switch(requestCode){
            case REQUEST_CODE_MAIN3ACTIVITY:
                if(resultCode== Activity.RESULT_OK){
                    int x = data.getIntExtra("EXTRA_INT",0);
                    tv.setText("Data received from Main3Activity:" + x);
                }
                break;
            ...
            default:
        }
    }
}
```

Intent: data returning (IV)

```
public class Main3Activity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main3);

        Button bt = (Button) findViewById(R.id.button5);
        bt.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent();
                intent.putExtra("EXTRA_INT",325);
                setResult(Activity.RESULT_OK,intent);
                finish();
            }
        });
    }
}
```

`startActivityForResult` deprecated

- Solution: Activity Results API

```
ActivityResultLauncher<Intent> mainActivity4Launcher = registerForActivityResult(
 new ActivityResultContracts.StartActivityForResult(),
 new ActivityResultCallback<ActivityResult>() {
   @Override
   public void onActivityResult(ActivityResult result) {
     if (result.getResultCode() == Activity.RESULT_OK) {
       // There are no request codes
       Intent data = result.getData();
       String msg = data.getStringExtra("EXTRA_MESSAGE");
       tv.setText("The message received from MainActivity4:\n\t" + msg);
     }
   }
 });

 Button button3 = (Button)findViewById(R.id.button3);
 button3.setOnClickListener(new View.OnClickListener() {
   public void onClick(View arg0) {
     Intent i = new Intent(MainActivity.this, MainActivity4.class);
   mainActivity4Launcher.launch(i);
   }
 });
```

Check: `TP04_02`

Activity Result API (I)

- `registerForActivityResult` - For registering the result callback
- `ActivityResultContract` - A contract specifying that an activity can be called with an input of type I and produce an output of type O.
- `ActivityResultCallback` - A type-safe callback to be called when an activity result is available.
  - It is a single method interface with an `onActivityResult` method that takes an object of the output type defined in the `ActivityResultContract`
- `ActivityResultLauncher` - A launcher for a previously-prepared call to start the process of executing an `ActivityResultContract`.

Activity Result API (II)

- `registerForActivityResult` takes an `ActivityResultContract` and an `ActivityResultCallback` and returns an `ActivityResultLauncher` which you'll use to launch the other activity.
- `launch` starts the process of producing the result.

Shared Preferences

What are `SharedPreferences`?

- A `SharedPreferences` object points to a file containing `key,value` pairs and provides simple methods to read and write them.
- Each `SharedPreferences` file is managed by the framework and can be private or shared.
- **They are accessible from anywhere within the app to either put data into the file or take data out of the file**.
- For primitive data types: booleans, floats, ints, longs, and strings.

Get Shared Preferences

- Use `getSharedPreferences` from Activity `Context`.

```
Context context = getActivity();
SharedPreferences sharedPref = context.getSharedPreferences("MyPrefs",
      Context.MODE_PRIVATE);
```

  - The first parameter is the preferences' name and the second parameter is the MODE.
    - `MODE_APPEND`: This will append the new preferences with the already existing preferences
    - `MODE_PRIVATE`: By setting this mode , the file can only be accessed using calling application
    - ...: some modes allow others apps to read or to write the preferences.

Write to Shared Preferences

- Create a `SharedPreferences.Editor` by calling `edit` on your `SharedPreferences`.
- Then call `commit` to save the changes.

```
/******* Create SharedPreferences *******/
SharedPreferences sharedPref = context.getSharedPreferences("MyPrefs", Context.
     MODE_PRIVATE);
SharedPreferences.Editor editor = sharedPref.edit();

/*************** Storing data as KEY/VALUE pair ******************/

editor.putBoolean("key_name1", true); // Saving boolean - true/false
editor.putInt("key_name2", "int value"); // Saving integer
editor.putFloat("key_name3", "float value"); // Saving float
editor.putLong("key_name4", "long value"); // Saving long
editor.putString("key_name5", "string value"); // Saving string

// Save the changes in SharedPreferences
editor.commit(); // commit changes
```

Read from Shared Preferences

- To retrieve values from a shared preferences file, call methods such as `getInt` and `getString`, providing the key for the value you want, and optionally a default value to return if the key is not present.
- Checking if a particular preference exists (is set) or not using the `contains` method on the `SharedPreferences`

```
/******* Create SharedPreferences *******/
SharedPreferences sharedPref = context.getSharedPreferences("MyPrefs", Context.
    MODE_PRIVATE);
/*************** Get SharedPreferences data ******************/
// If value for key not exist then return second param value - In this case null
sharedPref.getBoolean("key_name1", null); // getting boolean
sharedPref.getInt("key_name2", null); // getting Integer
sharedPref.getFloat("key_name3", null); // getting Float
sharedPref.getLong("key_name4", null); // getting Long
sharedPref.getString("key_name5", null); // getting String
/*************** Checking SharedPreferences data ******************/
boolean exists = sharedPref.contains("username");
```

Deleting keys and data

- `clear` method, marks in the editor to clear all values (data) from shared preferences
- `remove` method, marks in the editor that a preference value should be removed.

```
/******* Create SharedPreferences *******/
SharedPreferences sharedPref = context.getSharedPreferences("MyPrefs", Context.
     MODE_PRIVATE);
SharedPreferences.Editor editor = sharedPref.edit();

/************ Deleting Key value from SharedPreferences ****************/

editor.remove("key_name3"); // will delete key key_name3
editor.remove("key_name4"); // will delete key key_name4

// Save the changes in SharedPreferences
editor.commit(); // commit changes

/************ Clear all data from SharedPreferences ****************/

editor.clear();
editor.commit(); // commit changes
```

Commit changes

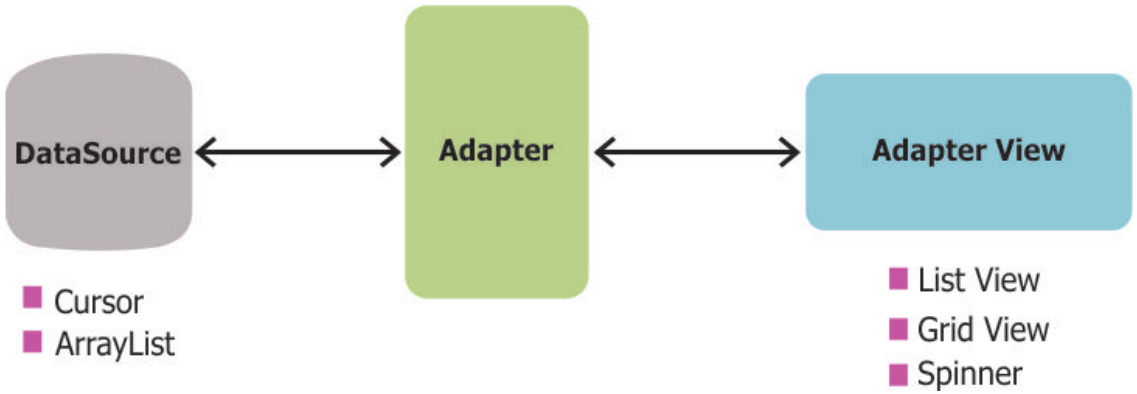
- In order to commit changes can be used two methods:
  - `apply`
    - This saves your data into memory immediately and saves the data to disk on a separate thread. So there is no chance of blocking the main thread (your app won't hang).
    - It is the preferred technique but has only been available since Gingerbread (API 9, Android 2.3).
  - `commit`
    - Calling this will save the data to the file however, the process is carried out in the thread that called it, stopping everything else until the save is complete. It returns true on successful completion, false on failure.
    - Use `commit` if you need confirmation of the success of saving your data or if you are developing for pre-Gingerbread devices.
    - It has been available since API 1

Check: `TP04_03`

Android Adapter

Adapters (I)

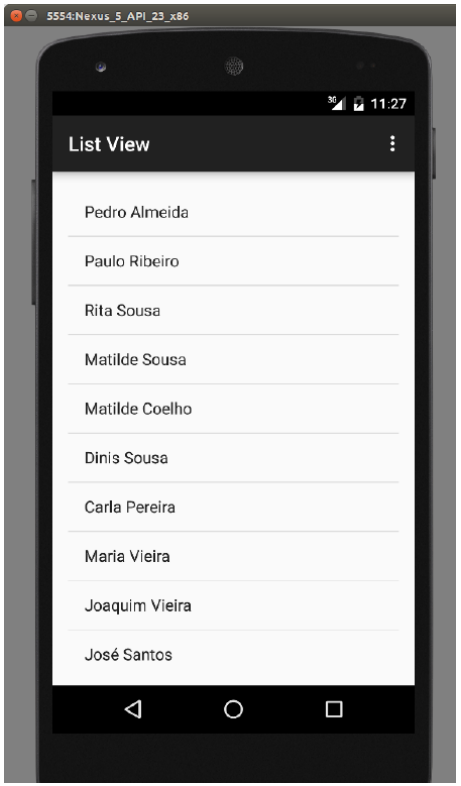
- Android's Adapter is described in the API documentation, as **a bridge** between an `AdapterView` and the underlying data for that `View`.
  - An `AdapterView` is a group of `View` components in Android that include the `ListView`, `Spinner`, and `GridView`.



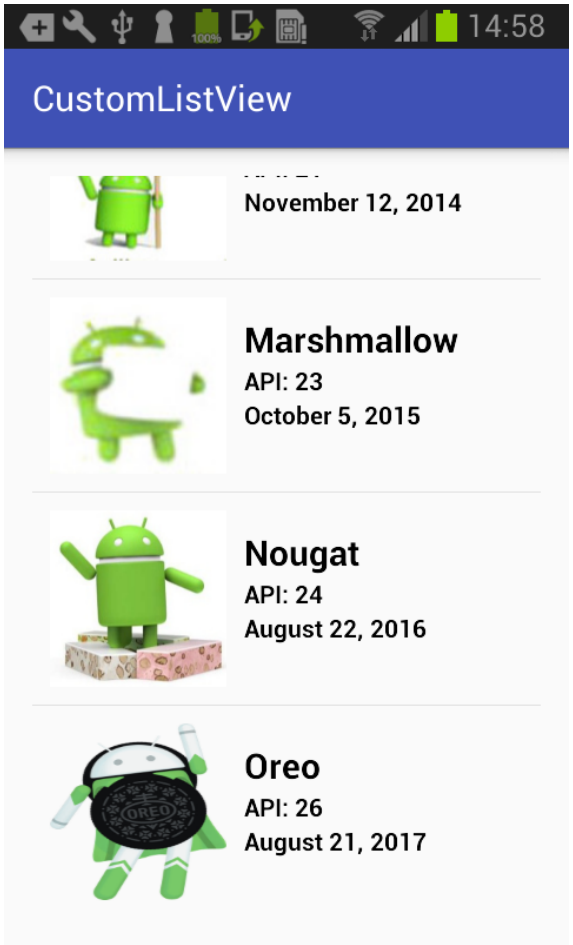
Adapters (II)

- For showing a **vertical list of scrollable items** it must be used a `ListView` which has data populated using an Adapter.
  - The simplest adapter to use is called an `ArrayAdapter` because the adapter converts an `ArrayList` of objects into `View` items loaded into the `ListView` container.

```
...
ListView lv;
ArrayList<String> list;
ArrayAdapter<String> adapter;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
lv = (ListView) findViewById (R.id.listView);
list = new ArrayList<String>();
String[] students = getResources().getStringArray(R.
      array.students);
Collections.addAll(list, students);
       adapter = new ArrayAdapter<String>(this,android.R.
      layout.simple_list_item_1,list);
       lv.setAdapter(adapter);
...
```

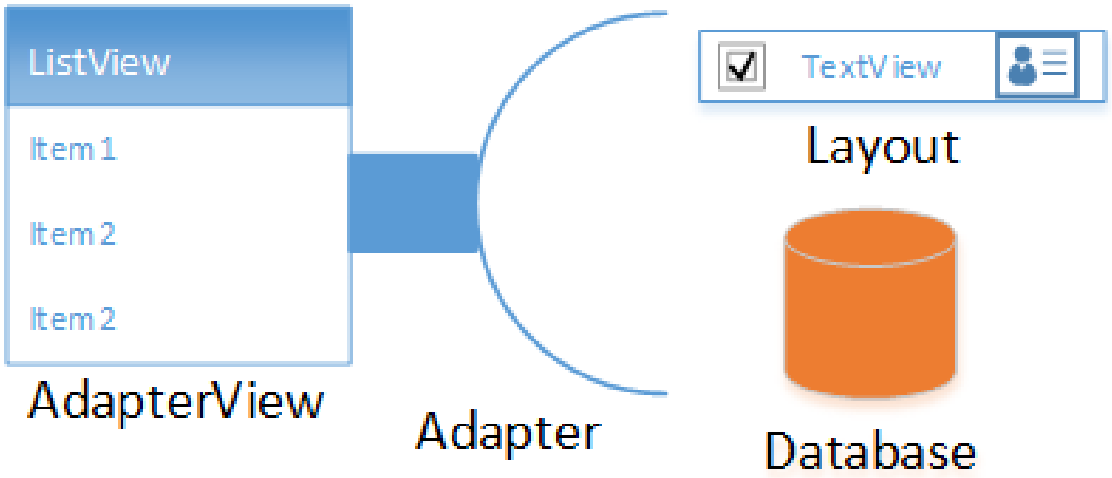


Custom Adapter (I)



Custom Adapter (II)

- In order to display a series of items into a list using a custom representation of the items:
  - Create a layout with a `ListView`.
  - Create a custom XML layout for each row.
  - Create a custom `Adapter` class, which could be a subclass of `BaseAdapter` class.



Custom Adapter: Create a layout with a `ListView`

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
...
  tools:context=".MainActivity">
  <ListView
    android:id="@+id/listView"
  .../>
</RelativeLayout>
```

```
public class MainActivity extends AppCompatActivity {
  ListView lv;
  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    lv = (ListView) findViewById(R.id.listView);
  }
}
```

Custom Adapter: Create a custom XML layout for each row

list_item.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
...>
  <ImageView
    android:id="@+id/version_icon"
    android:layout_width="100dp"
    android:layout_height="100dp"
    android:scaleType="fitXY"
    app:srcCompat="@drawable/noimage"/>
  <LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="10dp">
    <TextView
      android:id="@+id/version_name"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:textSize="20sp"
      android:textColor="@android:color/black"
      android:textStyle="bold" />
    ...
  </LinearLayout>
</LinearLayout>
```

Custom Adapter: Create a custom `Adapter` class, which could be a subclass of `BaseAdapter` class (I)

```
public class CustomAdapter extends BaseAdapter{
  Context context;
  int rowLayout;
  ArrayList<Data> data;
  public CustomAdapter (Context context, int rowLayout, ArrayList<Data> data){
    this.context = context;
    this.rowLayout = rowLayout;
    this.data = data;
  }
  @Override
  public int getCount() {
    return data.size();
  }
  @Override
  public Object getItem(int position) {
    return data.get(position);
  }
  @Override
  public long getItemId(int position) {
    return 0;
  }
  ...
}
```

Custom Adapter: Create a custom `Adapter` class, which could be a subclass of `BaseAdapter` class (II)

```
public class CustomAdapter extends BaseAdapter{
...
  @Override
  public View getView(int position, View convertView, ViewGroup parent) {
    View rowView;
    LayoutInflater inflater = (LayoutInflater)context.getSystemService(Context.
          LAYOUT_INFLATER_SERVICE);
    rowView = inflater.inflate(this.rowLayout, null);

    TextView name = (TextView) rowView.findViewById(R.id.version_name);
    ...
    ImageView icon = (ImageView) rowView.findViewById(R.id.version_icon);
    Data row = (Data)getItem(position);
    name.setText(row.getName());
    ...
    Drawable drawable = ContextCompat.getDrawable(this.context, row.getIcon());
    if(drawable == null) {
      drawable = ContextCompat.getDrawable(this.context, R.drawable.noimage);
    }
    icon.setImageDrawable(drawable);

    return rowView;
  }
}
```

Custom Adapter: Create a custom `Adapter` class, which could be a subclass of `BaseAdapter` class (III)

- `LayoutInflater` class is used to instantiate layout XML file into its corresponding `View` objects.




```
public class CustomAdapter extends BaseAdapter{
  ...
  @Override
  public View getView(int position, View convertView, ViewGroup parent) {
    View rowView;
    LayoutInflater inflater = (LayoutInflater)context.getSystemService(Context.
        LAYOUT_INFLATER_SERVICE);
    rowView = inflater.inflate(this.rowLayout, null);

    TextView name = (TextView) rowView.findViewById(R.id.version_name);
    ...
    ImageView icon = (ImageView) rowView.findViewById(R.id.version_icon);
    ...
    return rowView;
  }
}
```

Custom Adapter: Create a custom `Adapter` class, which could be a subclass of `BaseAdapter` class (IV)

```
public class MainActivity extends AppCompatActivity {
  ListView lv;

  ArrayList<Data> data;
  CustomAdapter adapter;

  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    lv = (ListView) findViewById(R.id.listView);

    data = getData();
    adapter = new CustomAdapter(this,R.layout.list_item, data);
    lv.setAdapter(adapter);

  }
  ...
}
```

Check: `TP04_04`

Threads & WorkerThreads

Different Ways Of Defining Threads In Java (I)

- Anonymous inner class

```
public static void main(String[] args){
  //Thread as an anonymous inner class
  new Thread()
  {
    @Override
    public void run()
    {
      System.out.println("Keep some task here.....");
    }
  }.start();
}
```

Different Ways Of Defining Threads In Java (II)

- As a `Thread` sublass

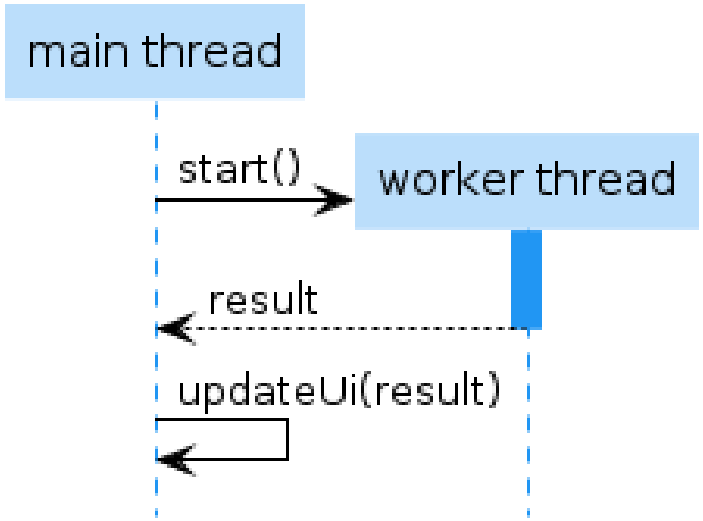
```
public class MyThread extends Thread {
  public MyThread(){
  }
  public void run(){
    System.out.println("MyThread running");
  }
    }
```

  - To create and start the above thread you can do like this:

```
MyThread myThread = new MyThread();
myTread.start();
```

- ...

UI Thread interacting with Worker Threads



- `runOnUIthread`
- `View.post`
- `Handler`

Check: `TP04_05`

Check: `TP04_06`

Check: `TP04_07`